

INSTITUCIÓN: Tecnológico Superior De
Lerdo

CARRERA: Ingeniería informática

MATERIA: Programación Orientada a
Objetos

TÍTULO: Portafolio de proyectos del
Semestre

ALUMNO: Ángel Ricardo Fernández Rivas

DOCENTE: Jesús Salas Marín

FECHA: 02 de Junio del año 2026

ÍNDICE

1. Introducción.
2. Desarrollo de proyectos .
 - 2.1 Proyecto 1: Examen Práctico Contra Reloj.
 - 2.2 Proyecto 2: Analizador de Consumo Energético en Servidores mediante Integración Numérica.
 - 2.3 Proyecto 3: Sistema Modular de Gestión de Usuarios y Roles Jerárquicos.
 - 2.4 Proyecto 4: Módulo de Gestión de inventarios y Análisis Estadístico mediante Arreglos Paralelos.
 - 2.5 Proyecto 5: Sistema de gestión e histórico de Bitácoras en Archivos de Texto Plano.
3. Reflexión Final.
4. Conclusiones Generales.
5. Referencias.

1. Introducción

El presente Portafolio de Proyectos representa una compilación exhaustiva y estructurada de las actividades de desarrollo de software y solución de problemas prácticos implementadas a lo largo del semestre en la materia de Programación Orientada a Objetos (POO). El propósito medular de este portafolio es proveer una evidencia sólida y transparente sobre la adquisición de competencias profesionales, la comprensión de arquitecturas lógicas y la evolución técnica lograda a través de la metodología de Aprendizaje Basado en Proyectos (ABPj).

A lo largo de las unidades de aprendizaje, las competencias desarrolladas se enfocaron de manera prioritaria en la capacidad de abstracción de problemas del entorno real, la conversión precisa de diagramas UML y especificaciones de requerimientos en código fuente ejecutable, la separación de responsabilidades lógicas y el aseguramiento del software mediante validaciones robustas y esquemas estructurados de manejo de excepciones. Dichas competencias fueron puestas en práctica mediante un enfoque multiplataforma y multilenguaje, abarcando las particularidades y la sintaxis avanzada de dos de los entornos con mayor demanda en el ámbito profesional actual: PHP y Python.

La Programación Orientada a Objetos se consolida como el paradigma dominante en la ingeniería de software debido a su idoneidad para atenuar la complejidad intrínseca de los sistemas modernos. Al modelar el software a través de entidades con atributos bien definidos y comportamientos aislados, la POO permite la creación de sistemas modulares, reutilizables y altamente escalables que reflejan con naturalidad el funcionamiento del mundo real. Complementando este rigor conceptual, el uso constante y organizado de la plataforma de control de versiones GitHub fue crítico durante el semestre. GitHub no solo sirvió como repositorio de almacenamiento, sino como el eje central de un flujo de trabajo organizado y transparente, fomentando hábitos profesionales de documentación sistemática, versión segura del código y buenas prácticas de trazabilidad en el ciclo de vida del desarrollo.

2. Desarrollo de Proyectos

2.1 Proyecto 1: Examen Práctico Contra Reloj - Modelado de Datos Logísticos (PHP)

Nombre del proyecto: Modelado Estricto de Datos Logísticos y Sensores Ambientales (Examen Contrarreloj - Corte 1).

Objetivo del proyecto: Medir la precisión técnica y la agilidad de desarrollo en la sintaxis estricta de PHP, enfocándose en la declaración de espacios de nombres, tipado estricto de atributos y aislamiento de visibilidad de datos sin adición de lógica de comportamiento.

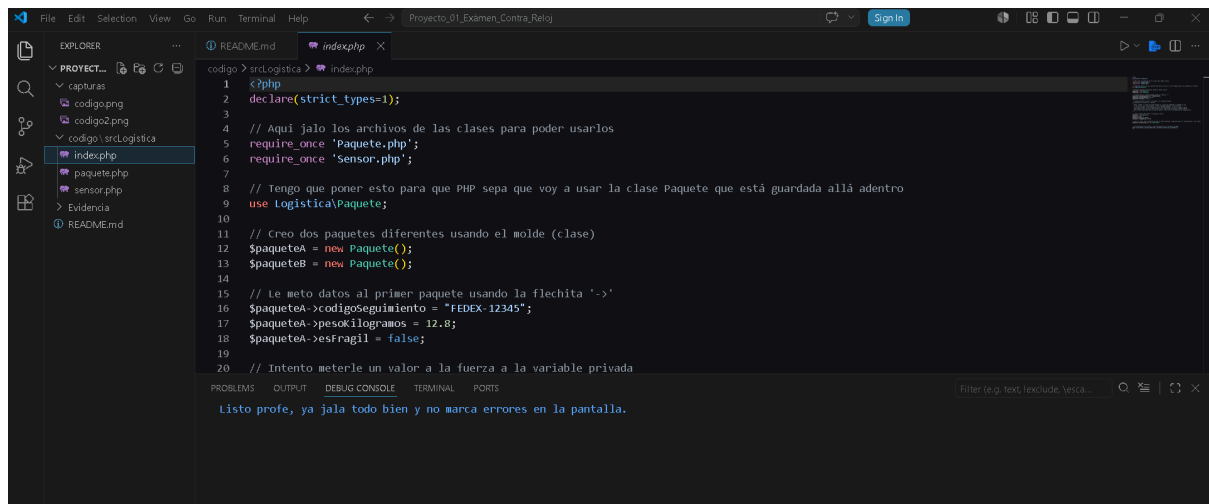
Problema planteado: La empresa de envíos rápidos FastDelivery requiere un molde de datos inmutable externo para el control logístico de sus envíos a través de un objeto Paquete. Paralelamente, se solicita un molde de control ambiental para invernaderos llamado Sensor. Ambos requerimientos debían implementarse en una estructura estricta de carpetas bajo un límite estricto de 30 minutos de codificación activa.

Tecnologías utilizadas: PHP, Namespaces, clases nativas de manejo del tiempo (\DateTime) y entorno de ejecución por consola.

Conceptos aplicados: Abstracción de datos, encapsulamiento mediante modificadores de visibilidad (public y private), Tipado Estricto (Type Hinting) y Modularización en el Sistema de Archivos.

Explicación del funcionamiento: El entorno se estructuró aislando la clase de transporte bajo la ruta física src/Logistica/Paquete.php bajo el espacio de nombres correspondientes. De forma paralela, la clase Sensor se dispuso en la raíz para simular integraciones externas. En el script de control index.php, se importaron mediante require_once y se instanciaron múltiples entidades. Para validar el encapsulamiento de datos, se incluyó intencionalmente un intento de asignación directa sobre la variable restringida costoInterno, provocando la interrupción inmediata del motor de PHP por violación de acceso, confirmando el correcto aislamiento del modelo.

Capturas del proyecto:



```
1 <?php
2 declare(strict_types=1);
3
4 // Aqui jalo los archivos de las clases para poder usarlos
5 require_once 'Paquete.php';
6 require_once 'Sensor.php';
7
8 // Tengo que poner esto para que PHP sepa que voy a usar la clase Paquete que está guardada allá adentro
9 use Logística\Paquete;
10
11 // Creo dos paquetes diferentes usando el molde (clase)
12 $paqueteA = new Paquete();
13 $paqueteB = new Paquete();
14
15 // Le meto datos al primer paquete usando la flechita '->'
16 $paqueteA->codigoSeguimiento = "FEDEX-12345";
17 $paqueteA->pesoKilogramos = 12.8;
18 $paqueteA->esFragil = false;
19
20 // Intento meterle un valor a la fuerza a la variable privada
```

Problemas OUTPUT DEBUG CONSOLE TERMINAL PORTS

Listo profe, ya jala todo bien y no marca errores en la pantalla.

Dificultades encontradas: La principal limitante fue la gestión óptima del tiempo (reloj de 30 minutos) para estructurar adecuadamente los directorios lógicos y evitar errores comunes al instanciar el objeto global del sistema DateTime dentro de un archivo que utiliza espacios de nombres personalizados.

Soluciones aplicadas: Se aplicó la sintaxis de barra invertida global (\DateTime) para indicarle al motor de PHP que busque la clase en la raíz del entorno global del lenguaje y no dentro del namespace App\Logística, logrando un código limpio y libre de advertencias en tiempo récord

2.2 Proyecto 2: Analizador de Consumo Energético en Servidores mediante Integración Numérica (PHP)

Nombre del proyecto: Monitor y Analizador de Consumo Energético en Servidores de Data Center (ABPj - Corte 2).

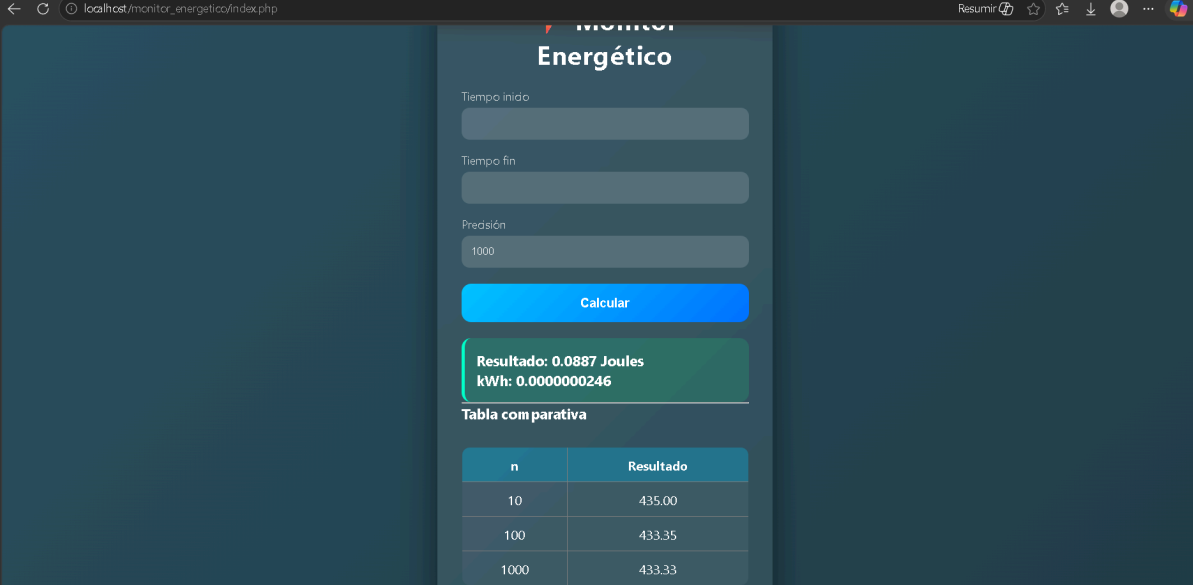
Objetivo del proyecto: Diseñar e implementar un módulo lógico modular y una interfaz web que estimen con alta precisión la Energía Total Consumida en Joules por un servidor en la nube a través de la aproximación matemática de la Regla del Trapecio, aplicando abstracción rigurosa y control defensivo de errores.

Problema planteado: El gasto eléctrico de un nodo de procesamiento cloud fluctúa de acuerdo con la carga de la CPU. Para calcular costos y emitir facturas precisas a los usuarios, se requiere calcular la integral definida de la función de potencia en Watts $P(t)$ en un intervalo temporal específico $[a, b]$. El sistema debe operar de forma segura ante datos incoherentes introducidos por el usuario y estructurarse como un componente reutilizable ("Caja Negra").

Tecnologías utilizadas: PHP orientado a objetos avanzado, HTML Estructural y CSS en hojas de estilo independientes para la maquetación visual del dashboard. Conceptos aplicados: Namespaces independientes, Encapsulamiento de variables internas, Abstracción procedimental, Manejo de Excepciones mediante bloques try-catch y análisis de costo computacional (Precisión vs. Rendimiento).

Explicación del funcionamiento: El motor lógico se encapsuló en la clase "IntegradorNumerico " dentro del namespace "App\Calculo". Al instanciar el objeto, el constructor evalúa la coherencia de los límites del intervalo y las iteraciones; si las condiciones fallan, dispara una excepción de sistema. Si los parámetros son correctos, el método público "calcularEnergiaTotal()" ejecuta un ciclo indexado que implementa de forma exacta la Regla Matemática del Trapecio sobre una función de potencia modelada internamente mediante una expresión robusta match() que discrimina entre perfiles de trabajo. El archivo raíz recopila los parámetros mediante peticiones seguras $$_POST$, delega el cálculo al objeto y despliega los resultados finales transformados en Joules y Kilovatios-hora (kWh)

Capturas del proyecto:



The screenshot shows a web browser window with the address bar displaying 'localhost/monitor_energetico/index.php'. The page title is 'Monitor Energético'. The interface includes three input fields: 'Tiempo inicio', 'Tiempo fin', and 'Presión'. The 'Presión' field contains the value '1000'. Below these fields is a blue 'Calcular' button. A green box displays the results: 'Resultado: 0.0887 Joules' and 'kWh: 0.0000000246'. Below the results is a table titled 'Tabla comparativa'.

n	Resultado
10	435.00
100	433.35
1000	433.33

Dificultades encontradas: Comprender la desconexión entre la interfaz gráfica del usuario y las variables matemáticas internas de la clase lógica, garantizando que el archivo frontal interactuara con el objeto tratándolo estrictamente como una "caja negra" sin conocer sus funciones internas.

Soluciones aplicadas: Se encapsularon los límites del cálculo de forma privada y se programó un menú dinámico basado en la estructura de control nativa match de PHP, permitiendo cambiar el comportamiento matemático interno fluidamente mediante un parámetro string limpio enviado desde el formulario.

2.3 Proyecto 3: Sistema Modular de Gestión de Usuarios y Roles Jerárquicos (Python)

Nombre del proyecto: Sistema de Autenticación y Control de Accesos mediante Herencia Jerárquica (ABPj - Corte 3).

Objetivo del proyecto: Desarrollar una solución integral en Python que explote las propiedades de la Herencia y el Polimorfismo, modelando roles corporativos diferenciados sobre una arquitectura modular distribuida en archivos independientes.

Problema planteado: Una plataforma informática requiere un subsistema para controlar los accesos y acciones de tres perfiles de usuarios: Administradores, Clientes e Invitados. Los perfiles comparten datos generales de registro pero sus flujos de ejecución, niveles de privilegios y respuestas operativas dentro del sistema de software deben estar completamente segregados.

Tecnologías utilizadas: Python, arquitectura de módulos importados, tipado dinámico e interfaz por línea de comandos (CLI).

Conceptos aplicados: Superclases y Subclases (Herencia Simple), Sobrescritura de Métodos, Polimorfismo Dinámico y reutilización de lógica constructora mediante la función nativa `super()`.

Explicación del funcionamiento: Se programó una superclase base en "usuario.py" que unifica los atributos esenciales "(nombre, email)". A partir de ella, se estructuraron las clases derivadas en archivos separados: "admin" introduce la variable `nivel_acceso`; "cliente" implementa un monedero de puntos de recompensa; e "invitado" restringe la cuenta a consultas pasivas. El polimorfismo se manifiesta de forma directa en el archivo principal, donde se inicializa una lista unificada de Python con instancias mixtas de usuarios; al recorrer dicha lista en un ciclo `for` y mandar llamar al método común "`acceso_sistema()`", cada objeto responde de forma independiente adaptando su salida a su naturaleza en tiempo de ejecución.

Capturas del proyecto:

```
usuario.py > Usuario > acceso_sistema
56 | | | | print(f"El invitado {self.nombre} No tiene permisos")
57
58 # Aplicación principal (menú)
59 def menu():
60     # Lista para guardar los objetos
61     lista_usuarios = []
62
63     # Creación de los objetos requeridos
64     admin1 = Admin("Pedro Martínez", "pedro@it.com", "Administrador")
65     cliente1 = Cliente("Martín Lopez", "martin@mail.com", 1500)
66     invitado1 = Invitado("Miguel Reyes", "miguel@invitado.com")
67     invitado2 = Invitado("Emmanuel Alemán", "emmanuel.com ")
68 ..

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

ERROR: El email 'emmanuel.com ' no es válido. Registro bloqueado.
- SISTEMA DE CONTROL DE USUARIOS -
Hola, soy Pedro Martínez. Mucho gusto
Nombre: Pedro Martínez | Email: pedro@it.com
ACCESO TOTAL: Bienvenido Administrador Pedro Martínez (Nivel de acceso: Administrador)
-----
Hola, soy Martín Lopez. Mucho gusto
Nombre: Martín Lopez | Email: martin@mail.com
ACCESO LIMITADO: Bienvenido Cliente Martín Lopez. Tienes 1500 puntos.
-----
Hola, soy Miguel Reyes. Mucho gusto
Nombre: Miguel Reyes | Email: miguel@invitado.com
ACCESO TEMPORAL: El invitado Miguel Reyes tiene permisos de solo lectura.
-----
Hola, soy Emmanuel Alemán. Mucho gusto
ERROR: No se pueden mostrar datos. Usuario no válido
El invitado Emmanuel Alemán No tiene permisos
-----
```

Dificultades encontradas: Adaptarse de forma abrupta al cambio de sintaxis de PHP a Python, lidiando con la obligatoriedad del parámetro de instancia self en la cabecera de todos los métodos de clase y la correcta invocación del constructor de la superclase.

Soluciones aplicadas: Se asimiló la filosofía de Python mediante la revisión detallada de su documentación técnica, implementando de forma precisa la función "super().__init__(nombre, email)" dentro de los bloques hijos para derivar limpiamente la inicialización de las propiedades compartidas.

2.4 Proyecto 4: Módulo de Gestión de Inventarios y Análisis Estadístico mediante Arreglos Paralelos (PHP)

Nombre del proyecto: Prototipo In-Memory para la Gestión y Análisis Estadístico de Inventarios Comerciales (Actividad Integradora).

Objetivo del proyecto: Utilizar arreglos unidimensionales en PHP estructurados de forma paralela para capturar, almacenar de forma temporal, procesar algebraicamente y presentar colecciones de información de artículos comerciales de forma limpia.

Problema planteado: Una microempresa de comercio electrónico requiere un módulo básico de inventario sin depender de infraestructura pesada de bases de datos relacionales. El sistema debe recopilar de forma masiva los nombres y precios de un mínimo de 5 artículos mediante un formulario web, calcular el volumen total monetario, el promedio general e identificar con precisión el producto de mayor costo y el más económico del lote.

Tecnologías utilizadas: HTML con controles nativos de validación, PHP y funciones integradas de análisis de arreglos ("array_sum()", "max()", "min()", "array_search()").

Conceptos aplicados: Matrices unidimensionales, Estructuras Paralelas Vinculadas por Índice, Paso de Parámetros Globales (\$_POST) y Renderizado Estructurado de Tablas Dinámicas.

Explicación del funcionamiento: El script de entrada index.php presenta un formulario cuyos campos de texto y número comparten nombres indexados. Al ser procesado por procesar.php, el servidor genera de forma automática dos vectores paralelos alineados por su clave posicional ("\$productos[]" y "\$precios[]"). En lugar de anclar ciclos de iteración tradicionales, el script consume las funciones matemáticas optimizadas de PHP para calcular los acumulados y utiliza "array_search()" para mapear el índice exacto del precio extremo, recuperando el nombre del producto asociado en la matriz hermana para finalmente imprimir los indicadores en una tabla limpia.

Capturas del proyecto:

Registro de Productos - Tienda en Línea

#	Nombre del Producto	Precio (\$)
1	Coca-cola	23
2	Sabritas	20
3	Pan francés	13
4	Lapiz	5
5	Pluma	7

Calcular y Procesar Inventario

Reporte Final de Inventario (Resultados)

Producto	Precio
Coca-cola	\$23.00
Sabritas	\$20.00
Pan francés	\$13.00
Lapiz	\$5.00
Pluma	\$7.00

Resumen de Análisis Financiero:

- Total de la venta potencial: \$68.00
- Promedio de precios: \$13.60
- Producto más caro: Coca-cola (\$23.00)
- Producto más barato: Lapiz (\$5.00)

[Registrar nuevos productos](#)

Dificultades encontradas: El riesgo de asimetría o desalineación de datos; si por algún error en la transferencia o alteración de la petición un arreglo llegase a tener menos elementos que el otro, el cálculo estadístico se corromperá por completo mapeando precios a productos equivocados.

Soluciones aplicadas: Se insertó un filtro de control previo en el backend utilizando la función condicional "count(\$productos) === count(\$precios)", cancelando la ejecución e informando un error controlado al usuario en caso de detectar discrepancias estructurales.

2.5 Proyecto 5: Sistema de Gestión e Histórico de Bitácoras en Archivos de Texto Plano (PHP)

Nombre del proyecto: Módulo Digitalizado de Control de Incidencias mediante Persistencia en Archivos Planos (ABP).

Objetivo del proyecto: Dominar las técnicas lógicas de persistencia física de datos en el servidor mediante la apertura, escritura segura en modo anexado (append) y lectura estructurada de archivos de texto plano, implementando filtros defensivos contra inyección de scripts.

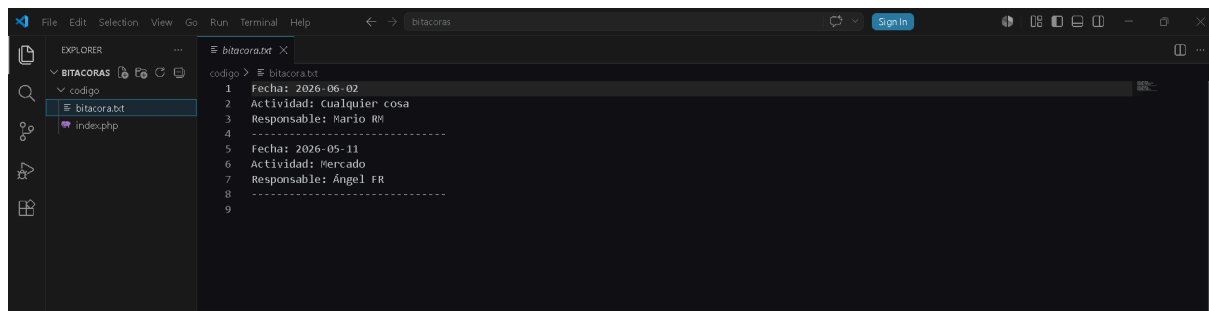
Problema planteado: Una corporación de seguridad privada busca eliminar las Bitácoras de incidencias en papel mediante una herramienta ligera que no requiera el soporte ni los costos operativos de un motor de bases de datos. El sistema debe asegurar que las nuevas inserciones se añaden al final del histórico existente sin riesgo de sobrescribir los registros del pasado.

Tecnologías utilizadas: PHP, Funciones de Entrada/Salida de archivos nativas (file_put_contents(), file_get_contents()), sanitizadores de cadenas y etiquetas contenedoras HTML.

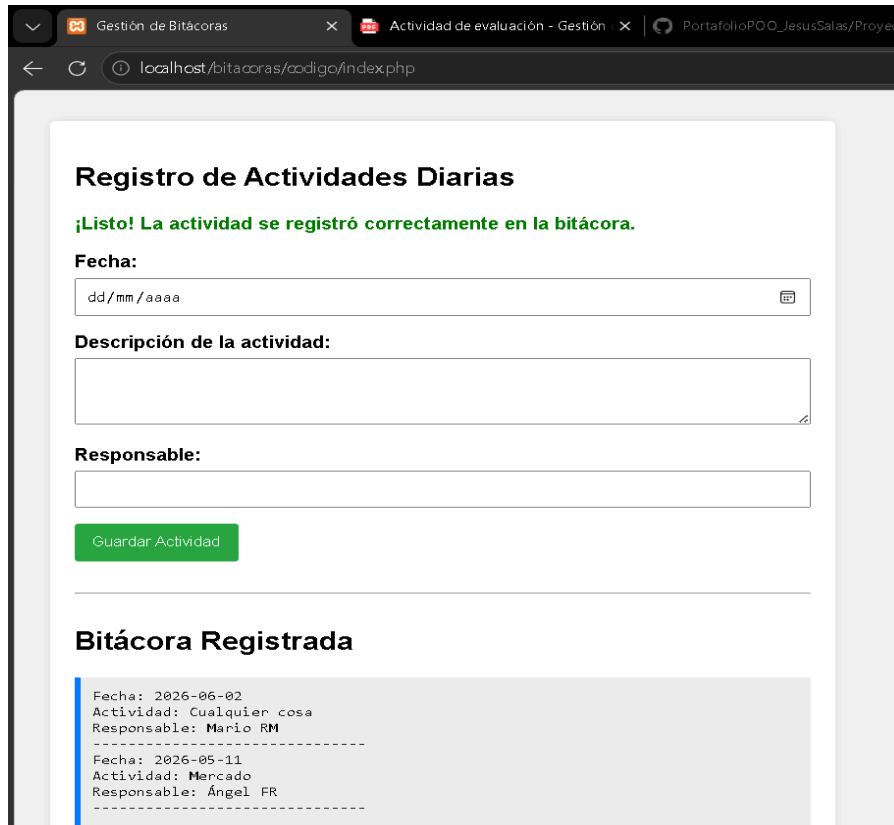
Conceptos aplicados: Persistencia de Datos, Flags de Control de Archivos (FILE_APPEND), Prevención de Ataques Cross-Site Scripting (XSS) y manipulación de streams de texto.

Explicación del funcionamiento: La interfaz unificada en index.php procesa de forma interna las dos caras del problema. Al llenarse los datos de la actividad (Descripción, Responsable, Fecha), el backend valida que las variables estén libres de espacios en blanco mediante "trim()" y limpia el texto con la función "htmlspecialchars()". Posterior a esto, formatea la información en una cadena estructurada y la inyecta al archivo físico "bitacora.txt" por medio de "file_put_contents()" configurada con el parámetro constante "FILE_APPEND". Inmediatamente después, el segundo bloque del script abre el mismo archivo en modo de lectura, fragmenta los registros y los imprime de forma limpia dentro de una lista ordenada estructurada en el frontend.

Capturas del proyecto:



```
1 Fecha: 2026-06-02
2 Actividad: Cualquier cosa
3 Responsable: Mario RM
4 -----
5 Fecha: 2026-05-11
6 Actividad: Mercado
7 Responsable: Ángel FR
8 -----
9
```



Registro de Actividades Diarias

¡Listo! La actividad se registró correctamente en la bitácora.

Fecha:

Descripción de la actividad:

Responsable:

Bitácora Registrada

```
Fecha: 2026-06-02
Actividad: Cualquier cosa
Responsable: Mario RM
-----
Fecha: 2026-05-11
Actividad: Mercado
Responsable: Ángel FR
-----
```

Dificultades encontradas: La vulnerabilidad crítica de seguridad informática que se presenta si un operador malintencionado introduce scripts de JavaScript en los campos de texto del formulario, lo que provocaría la ejecución involuntaria de código dañino al renderizar la bitácora histórica en el navegador de los supervisores.

Soluciones aplicadas: Se implementó un blindaje estricto aplicando de manera sistemática la función de conversión `htmlspecialchars($input)`, lo que deshabilita la ejecución de cualquier etiqueta HTML o script transformándola en texto plano inofensivo antes de guardarlo en el disco físico del servidor.

3. Reflexión Final

Todo lo práctico a lo largo de todos estos proyectos ha representado una experiencia de aprendizaje retadora y sumamente enriquecedora, forzándome a evaluar mis conocimientos de resolución de problemas desde la óptica de un desarrollador de software profesional.

¿Cuál proyecto fue más difícil? Sin lugar a dudas, el Analizador de Consumo Energético en Servidores. El desafío no se limitó a la codificación de la sintaxis, sino a la correcta abstracción matemática de la Regla del Trapecio combinada con la exigencia de aislar la lógica numérica de la interfaz web mediante un esquema estricto de manejo de excepciones y Namespaces. Aunque hacerlo en equipo ayudó, sin duda sí nos llevó algo de tiempo terminarlo para que el resultado final fuera el mejor posible.

¿Cuál fue más útil? El desarrollo del Sistema de Gestión de Bitácoras en Archivos de Texto. En la práctica real de la Ingeniería Informática, el análisis, generación y auditoría de archivos de registro (logs) en el servidor es una tarea cotidiana e indispensable que todo ingeniero debe dominar a la perfección sin depender necesariamente de sistemas gestores de bases de datos.

¿Qué habilidad mejoró más? Mi capacidad de depuración (debugging) y la visión estructurada de la arquitectura de software. Pasar de escribir scripts lineales y desorganizados a planificar una estructura modular limpia en carpetas y clases independientes, transformó positivamente mi lógica de programación.

¿Qué aprendiste sobre programación orientada a objetos? Comprendí que la POO ayuda mucho a la agrupación de variables. Conceptos como el encapsulamiento y la abstracción actúan como un blindaje corporativo que protege los datos esenciales de una empresa y facilitan el mantenimiento del software a largo plazo.

¿Qué aprendiste sobre trabajo organizado? Aprendí que escribir código funcional es solo la mitad del trabajo. La correcta maquetación de los archivos, la validación de datos en el backend y el control exhaustivo de versiones mediante repositorios remotos en GitHub constituyen la verdadera diferencia entre un programador amateur y un Ingeniero Informático profesional.

4. Conclusiones Generales

La culminación exitosa de los cinco proyectos presentados en este portafolio documenta un crecimiento técnico y analítico de gran valor. El progreso es evidente al observar la evolución del flujo de trabajo: se inició con el dominio básico de la sintaxis y el tipado estricto en PHP durante pruebas contra reloj, para posteriormente alcanzar el diseño avanzado de componentes aislados basados en modelos matemáticos integrales, la implementación de herencia y polimorfismo dinámico en lenguajes alternativos de alta demanda como Python, y la manipulación segura de persistencia física en archivos de texto plano.

La adopción de la metodología de Aprendizaje Basado en Proyectos (ABPj) demostró ser una estrategia pedagógica invaluable en nuestra formación profesional. Esta metodología nos obliga a salir de los entornos teóricos tradicionales para resolver problemas y necesidades reales del sector industrial, tales como la estimación precisa de costos de facturación en infraestructuras de la nube (estilo Amazon Web Services o Google Cloud), la optimización in-memory de catálogos comerciales o el diseño de sistemas de seguridad auditables.

Finalmente, se concluye que la documentación detallada del software y la adopción constante de herramientas profesionales de control de versiones como GitHub no constituyen procesos secundarios ni opcionales dentro del flujo de desarrollo. Estas herramientas representan los pilares éticos y profesionales de la ingeniería que garantizan la transparencia, el orden, la colaboración limpia y la continuidad a largo plazo de cualquier proyecto tecnológico en el mundo productivo actual.

5. Referencias Bibliográficas y Manuales Técnicos

1. **Documentación Oficial de PHP (PHP Manual).**
2. **Documentación Oficial de Python 3 (Python Docs):** Especificaciones del lenguaje sobre el modelo de clases, inicialización de objetos, herencia simple, herencia múltiple y el comportamiento polimórfico de las listas dinámicas. Disponible en: <https://docs.python.org/3/>.
3. **W3Schools Online Web Tutorials:** Guías de referencia y estándares prácticos para la maquetación de formularios validados en HTML, hojas de estilo CSS3 para entornos corporativos y manejo básico de arreglos paralelos. Disponible en: <https://www.w3schools.com/>.
4. **Manuales y Programas de Estudio Institucionales:** Temario formal de la asignatura Programación Orientada a Objetos, División de Ingeniería Informática, Instituto Tecnológico Superior de Lerdo, Coahuila/Durango, ciclo escolar 2026.
5. **Videos de Youtube:** Videos como la importancia de la creación de un README para mis proyectos y estructura de los repositorios, también para entender cierta lógica de los códigos.
6. **IA:** Herramientas como Gemini que utilicé para los errores de sintaxis en la terminal de Git, comprender conceptos y la estructura de los README.